Práctica 3: Concertpass

Entrega: 29 de octubre de 2026

Equipo: Luis Guillermo Hernandez Casillas, Carlos Emiliano Olmedo Navarro, Isaian Ayala García | Materia: Desarrollo en la Nube | Repositorio: https://github.com/luishdezc/concertpass.git | Video: https://youtu.be/5RQGlADkyH4

Descripción del proyecto

Concertpass es un sistema de reservación de boletos para conciertos diseñado para aguantar picos de demanda sin vender más boletos que el aforo. Lo forman tres microservicios en Python, desplegados como pods en un cluster de Amazon EKS de dos nodos t3.medium creado con eksctl en AWS Academy.

ticket-api recibe las solicitudes: POST /reserve guarda la reservación en RDS PostgreSQL con estado pending, la encola en una cola SQS FIFO y regresa el reservation_id; GET /reservation/{id} devuelve el estado actual. Se expone fuera del cluster con un Service NodePort. ticket-worker hace polling a la cola; por cada mensaje lee la capacidad de la sección en DynamoDB, cuenta en RDS los boletos vendidos dentro de una transacción con FOR UPDATE y marca la reservación como booked o rejected. Si queda booked, llama a ticket-checkout, que simula un procesador de pagos y responde 200 OK, y la marca como confirmed.

DynamoDB guarda la definición de los eventos (nombre, fecha, recinto y capacidad por sección) y solo se lee en runtime. Las credenciales de RDS viven en Secrets Manager: External Secrets Operator las copia a un Secret de Kubernetes que llega a los pods como variables de entorno, y el resto de la configuración va en un ConfigMap. Las imágenes se publican en Amazon ECR, y toda la infraestructura se levanta con setup.sh (bash y Terraform) y se destruye con teardown.sh.

En la prueba de alta demanda, 30 solicitudes simultáneas para 5 boletos VIP, con tres réplicas del worker, terminaron en 5 confirmed y 25 rejected, sin sobreventa.

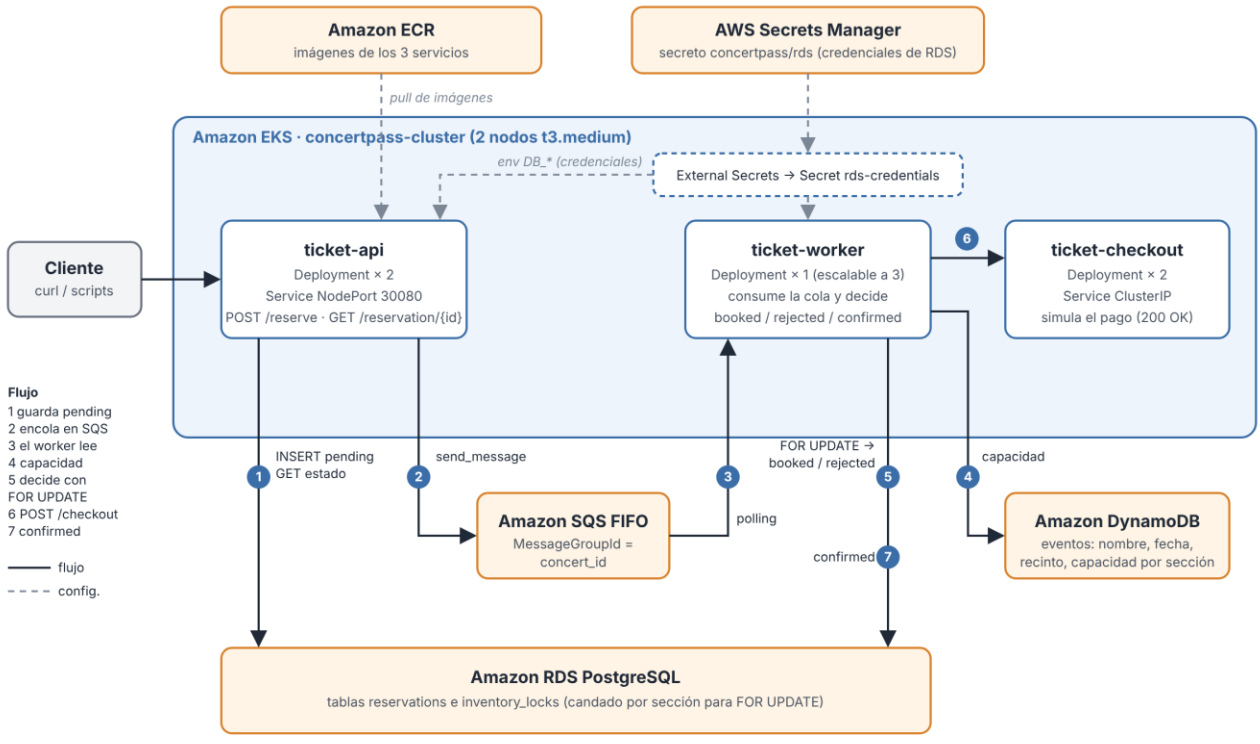


Figura 1. Arquitectura de Concertpass y flujo de una reservación.

Preguntas

1. ¿Para qué sirve SQS FIFO con MessageGroupId y cómo se usa en esta actividad?

Una cola SQS FIFO entrega los mensajes en el orden en que se enviaron y descarta duplicados dentro de una ventana de 5 minutos. El orden se garantiza dentro de cada MessageGroupId: mientras un mensaje de un grupo está en proceso, SQS no entrega el siguiente de ese grupo, y los grupos distintos avanzan en paralelo.

En Concertpass, ticket-api usa concert_id como MessageGroupId y reservation_id como MessageDeduplicationId. Así, las reservaciones de un mismo concierto se atienden en orden de llegada (el primero en reservar es el primero en obtener boleto), mientras que conciertos distintos se procesan al mismo tiempo. La cola además desacopla la recepción del procesamiento: el API responde 202 en milisegundos aunque el worker vaya atrasado. Si el worker falla, el mensaje regresa a la cola al vencer el visibility timeout; tras cinco intentos pasa a una cola de mensajes fallidos (DLQ) para no bloquear al resto del grupo.

2. ¿Qué problema resuelve FOR UPDATE y cómo garantiza consistencia? ¿Cuál es el trade-off entre concurrencia y consistencia?

Resuelve la condición de carrera de "leer y después escribir": sin bloqueo, dos workers pueden leer "quedan 2 boletos" al mismo tiempo y ambos vender. FOR UPDATE bloquea las filas leídas hasta el COMMIT o ROLLBACK; otro worker que intente bloquearlas espera y, al continuar, ve los datos ya actualizados.

El SQL sugerido no funcionó tal cual: PostgreSQL rechaza FOR UPDATE junto con SUM(), y reescrito como subconsulta solo bloquea las filas ya vendidas, así que con el concierto vacío no bloquea nada. Lo medimos con 30 workers simultáneos y 10 boletos: se vendieron 30. Nuestra solución hace FOR UPDATE sobre una fila de candado por concierto y sección (tabla inventory_locks) antes de sumar los vendidos; con ella se vendieron exactamente 10 y se rechazaron 20. Si no hay espacio, se hace ROLLBACK y solo esa reservación pasa a rejected.

El trade-off: las ventas de una misma sección quedan en fila, una a la vez. Agregar réplicas del worker o del API no acelera un concierto popular; el cuello de botella es la lógica de inventario (leer, bloquear, escribir), no la recepción de requests. Secciones y conciertos distintos sí avanzan en paralelo. Elegimos consistencia sobre concurrencia porque una sobreventa cuesta más que unos milisegundos de espera.

3. ¿Qué beneficios te da usar EKS en este caso?

EKS nos da Kubernetes con el control plane administrado por AWS. Cada servicio escala por separado: para la prueba de carga subimos el worker a tres réplicas con un solo comando, sin tocar ticket-api ni ticket-checkout. Kubernetes reinicia los pods que fallan su liveness probe y solo manda tráfico a los que pasan el readiness probe. El DNS interno conecta al worker con ticket-checkout (http://ticket-checkout:8080) sin IPs fijas, y los Services NodePort y ClusterIP separan lo público de lo interno.

La configuración se inyecta con ConfigMaps y Secrets, así que la misma imagen de ECR corre en cualquier ambiente, y los manifiestos versionados en Git describen todo el despliegue. EKS también se integra con IAM, ECR y Secrets Manager. Como limitación, AWS Academy no permite crear roles IAM, así que todos los pods comparten LabRole; en una cuenta normal usaríamos IRSA o Pod Identity para dar a cada pod solo los permisos que necesita.